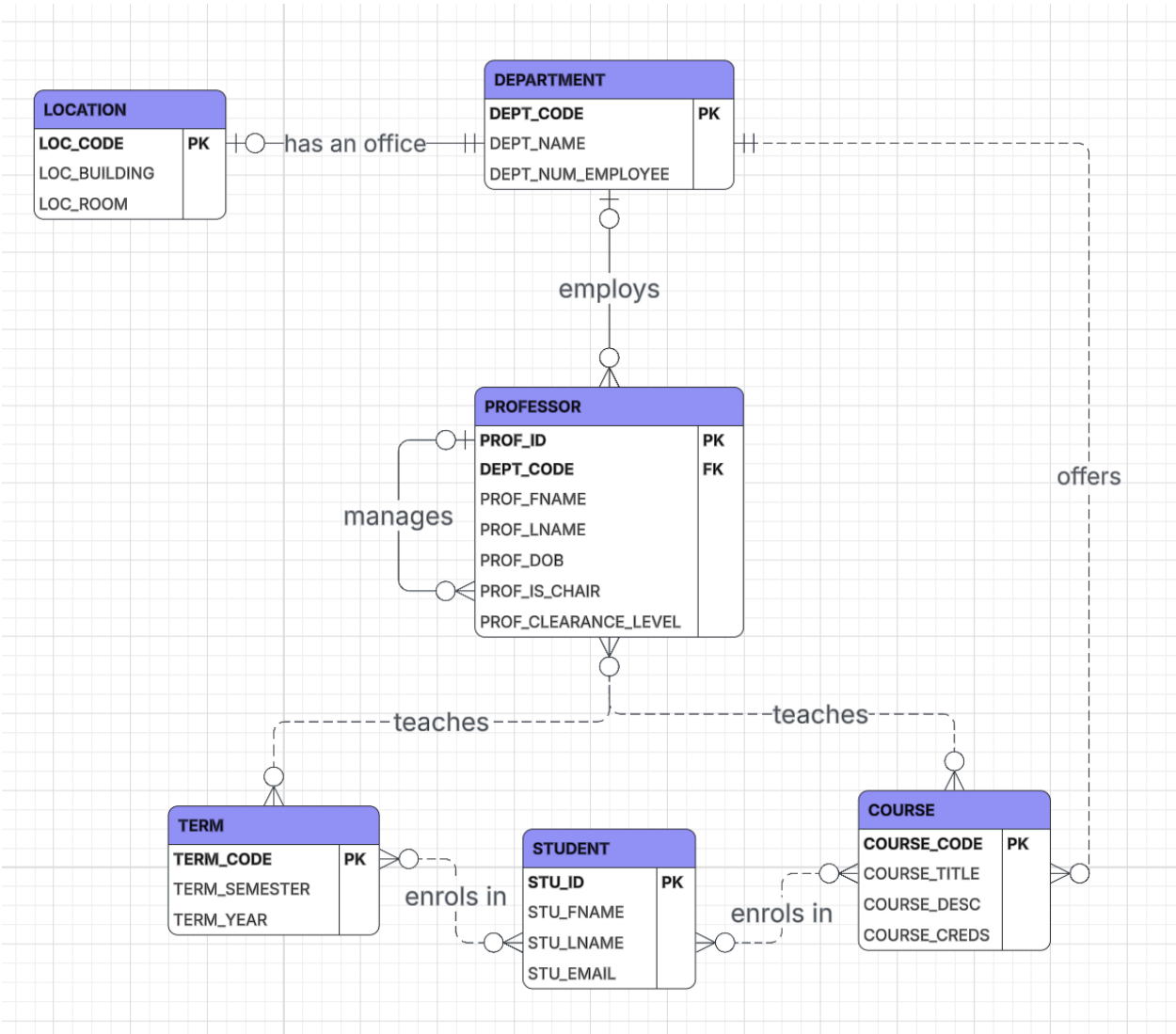
Revised EERD:



Logical Relational Schema

LOCATION (**LOC_CODE**, LOC_BUILDING, LOC_ROOM)

DEPARTMENT (**DEPT_CODE**, DEPT_NAME, DEPT_NUM_EMPLOYEE, *LOC_CODE*)

- *LOC_CODE FK references LOCATION*

PROFESSOR (**PROF_ID**, *DEPT_CODE*, PROF_FNAME, PROF_LNAME, PROF_DOB, PROF_IS_CHAIR, PROF_CLEARANCE_LEVEL, *MANAGER_ID*)

- *DEPT_CODE FK references DEPARTMENT*
- *MANAGER_ID FK references PROFESSOR (Self-referencing relationship for "manages")*

TERM (**TERM_CODE**, TERM_SEMESTER, TERM_YEAR)

COURSE (**COURSE_CODE**, *DEPT_CODE*, COURSE_TITLE, COURSE_DESC, COURSE_CREDS)

- *DEPT_CODE FK references DEPARTMENT (from the "offers" relationship)*

STUDENT (**STU_ID**, STU_FNAME, STU_LNAME, STU_EMAIL)

CLASS_SECTION (Associative table for Professor teaches Course in a Term)

- (**PROF_ID, COURSE_CODE, TERM_CODE**)

ENROLLMENT (Associative table for Student enrolling in Course/Term)

- (**STU_ID, COURSE_CODE, TERM_CODE**)

Final Relational Schema

```sql
-- 1. Location Table
CREATE TABLE LOCATION (
    LOC_CODE VARCHAR(10) PRIMARY KEY,
    LOC_BUILDING VARCHAR(100) NOT NULL,
    LOC_ROOM VARCHAR(20) NOT NULL,
    -- Ensure room/building combinations are unique
    CONSTRAINT UNIQUE_ROOM_LOC UNIQUE (LOC_BUILDING, LOC_ROOM)
);

-- 2. Department Table
CREATE TABLE DEPARTMENT (
    DEPT_CODE VARCHAR(10) PRIMARY KEY,
    DEPT_NAME VARCHAR(100) NOT NULL UNIQUE,
    DEPT_NUM_EMPLOYEE INTEGER DEFAULT 0
        CONSTRAINT POSITIVE_EMPLOYEES CHECK (DEPT_NUM_EMPLOYEE >= 0),
    LOC_CODE VARCHAR(10) NOT NULL,
    FOREIGN KEY (LOC_CODE) REFERENCES LOCATION(LOC_CODE) ON DELETE CASCADE
);

-- 3. Professor Table
CREATE TABLE PROFESSOR (
    PROF_ID SERIAL PRIMARY KEY,
    DEPT_CODE VARCHAR(10) NOT NULL,
    PROF_FNAME VARCHAR(50) NOT NULL,
    PROF_LNAME VARCHAR(50) NOT NULL,
    PROF_DOB DATE NOT NULL
        CONSTRAINT PROF_AGE_CHECK CHECK (PROF_DOB < CURRENT_DATE - INTERVAL '18 years'),
    PROF_IS_CHAIR BOOLEAN DEFAULT FALSE,
    PROF_CLEARANCE_LEVEL INTEGER NOT NULL
        CONSTRAINT VALID_CLEARANCE CHECK (PROF_CLEARANCE_LEVEL BETWEEN 1 AND 5),
    MANAGER_ID INTEGER,
    FOREIGN KEY (DEPT_CODE) REFERENCES DEPARTMENT(DEPT_CODE),
    FOREIGN KEY (MANAGER_ID) REFERENCES PROFESSOR(PROF_ID) ON DELETE SET NULL
);

-- 4. Term Table
CREATE TABLE TERM (
    TERM_CODE VARCHAR(10) PRIMARY KEY,
    TERM_SEMESTER VARCHAR(20) NOT NULL,
    TERM_YEAR INTEGER NOT NULL,
    -- Logic constraint: Term code often contains year/sem info
    CONSTRAINT VALID_SEMESTER CHECK (TERM_SEMESTER IN ('Fall', 'Winter', 'Spring',
'Summer')),
    CONSTRAINT VALID_YEAR CHECK (TERM_YEAR >= 2000)
);
```

```sql
-- 5. Course Table
CREATE TABLE COURSE (
    COURSE_CODE VARCHAR(10) PRIMARY KEY,
    DEPT_CODE VARCHAR(10) NOT NULL,
    COURSE_TITLE VARCHAR(100) NOT NULL,
    COURSE_DESC TEXT,
    COURSE_CREDS NUMERIC(3,1) NOT NULL
        CONSTRAINT VALID_CREDITS CHECK (COURSE_CREDS > 0 AND COURSE_CREDS <= 12),
    FOREIGN KEY (DEPT_CODE) REFERENCES DEPARTMENT(DEPT_CODE)
);

-- 6. Student Table
CREATE TABLE STUDENT (
    STU_ID SERIAL PRIMARY KEY,
    STU_FNAME VARCHAR(50) NOT NULL,
    STU_LNAME VARCHAR(50) NOT NULL,
    STU_EMAIL VARCHAR(100) NOT NULL UNIQUE
        CONSTRAINT VALID_EMAIL CHECK (STU_EMAIL LIKE '%@%.%')
);

-- 7. Teaches (Associative table)
CREATE TABLE TEACHES (
    PROF_ID INTEGER NOT NULL,
    COURSE_CODE VARCHAR(10) NOT NULL,
    TERM_CODE VARCHAR(10) NOT NULL,
    PRIMARY KEY (PROF_ID, COURSE_CODE, TERM_CODE),
    FOREIGN KEY (PROF_ID) REFERENCES PROFESSOR(PROF_ID),
    FOREIGN KEY (COURSE_CODE) REFERENCES COURSE(COURSE_CODE),
    FOREIGN KEY (TERM_CODE) REFERENCES TERM(TERM_CODE)
);

-- 8. Enrolls In (Associative table)
CREATE TABLE ENROLLS_IN (
    STU_ID INTEGER NOT NULL,
    COURSE_CODE VARCHAR(10) NOT NULL,
    TERM_CODE VARCHAR(10) NOT NULL,
    ENROLL_DATE DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (STU_ID, COURSE_CODE, TERM_CODE),
    FOREIGN KEY (STU_ID) REFERENCES STUDENT(STU_ID),
    FOREIGN KEY (COURSE_CODE) REFERENCES COURSE(COURSE_CODE),
    FOREIGN KEY (TERM_CODE) REFERENCES TERM(TERM_CODE)
);
```

Mapping Notes

1. Handling M:N Relationships

Two significant **M:N** relationships: **Student to Course** ("enrols in") and **Professor to Course** ("teaches").

Note: **M:N** relationships cannot be implemented directly in relational models with a only one foreign key because a single cell cannot hold multiple values.

- **Implementation:** I resolved these by creating **Associative (Junction) Tables**: **ENROLLS_IN** and **TEACHES**.
- **Composite Primary Keys:** For **ENROLLS_IN** - using a composite primary key consisting of (**STU_ID**, **COURSE_CODE**, **TERM_CODE**).
    - Ensures a student can't be enrolled in the same course twice in the same term, while successfully linking the three entities (Student, Course, and Term).

2. Handling Weak Entities

In this specific schema, entities like **COURSE** or **PROFESSOR** function as **Strong Entities** because they have unique identifiers (**COURSE_CODE**, **PROF_ID**). However, the associative tables like **TEACHES** can be viewed through the lens of **Weak Entities** (specifically identifying relationships), as their existence depends entirely on the parent entities they link.

- **Implementation:** Ensured **Referential Integrity** by using **FOREIGN KEY** constraints that point back to the strong "owner" entities.
- **Cascade Logic:** To maintain the "weak" dependency - utilized **ON DELETE CASCADE** in certain areas. This makes sure that if a parent record (like a specific **COURSE_CODE**) is removed, the dependent "teaching" or "enrollment" records also gets removed to prevent orphaned data in the db.